ChatGPT

Coding Exam Cheatsheet (2019–2024)

2019 Exam (Data Analysis & Machine Learning)

- **Pandas Data Analysis:** Read data with pandas (e.g. `pd.read_csv`), parse dates, and filter or group data by date/time. For example, converting upload dates to Year and grouping by year to count videos [1] [2]. Use **groupby** to aggregate (e.g. count videos per year or per channel). Basic descriptive analysis may involve bootstrapping with NumPy (sampling with replacement to compute confidence intervals of means) [3] [4]. Plotting is done with Matplotlib (line charts for trends over years, etc.).

- **Machine Learning Workflow:** Prepare features and target using pandas and scikit-learn:

- One-hot encode categorical variables (e.g. channel names) with `pd.get_dummies` [5] [6].
- Split data into train/test sets with `train_test_split` [7] [2].
- Train models like Ridge regression and Logistic regression:
  - Use `GridSearchCV` to tune hyperparameters (e.g. α for Ridge) with cross-validation [8].
  - Evaluate regression performance via mean absolute error [9].
  - For classification, use `LogisticRegressionCV` or `LogisticRegression` with cross-validation to find optimal regularization C [10] [11].

- **Example:** Training a Ridge regressor to predict view counts, tuning regularization:

```
ridge = Ridge()
param_grid = {'alpha': [0.001, 0.01, 0.1]}
ridge_cv = GridSearchCV(ridge, param_grid, cv=3)
ridge_cv.fit(X_train, y_train)
best_alpha = ridge_cv.best_params_['alpha']
```

(Performs 3-fold CV to select α) [8]. After training, compute errors like `mean_absolute_error(y_test, ridge_cv.predict(X_test))` [12].

- **Scikit-Learn Metrics:** Use `mean_absolute_error`, `mean_squared_error` for regression, and for classification use accuracy or the `.score` method on test data [13]. In 2019, a logistic model predicting high-vs-low views achieved ~74.8% accuracy [14].

- **Network Analysis with NetworkX:** Build graphs from data (e.g. channels as nodes, similarity edges). In 2019 Task C, an adjacency matrix was provided and loaded into a NetworkX graph [15]. Common graph algorithms:

- Compute graph **diameter** (longest shortest-path). Use `nx.diameter(G)` if graph is connected, or handle exceptions if not (the 2019 solution noted potential errors if graph not fully connected) [16].
- Perform community detection, e.g. Kernighan–Lin bisection via `nx.algorithms.community.kernighan_lin_bisection(G, seed=42)` to split the graph and then count category distribution in communities [17] [18].
- Measure connectivity: e.g. compare two similarity graphs' diameters to infer diversity in content (2019 compared graphs based on video tags vs descriptions) and compute probabilities like P(community|category) to quantify clustering quality [19] [18].

2020 Exam (Text Classification & Network Analytics)

- **Text Classification (Logistic Regression):** Use scikit-learn's text processing and linear models to predict categories. Build a Pipeline combining `CountVectorizer` (bag-of-words) and `TfidfTransformer` with a classifier [20]. For example, to predict Wikipedia article topics from text, the exam used an SGDClassifier (logistic loss, 5 epochs) inside a Pipeline:

```
text_clf = Pipeline([
    ('vect', CountVectorizer()),
    ('tfidf', TfidfTransformer()),
    ('clf', SGDClassifier(loss='log', max_iter=5, tol=None,
random_state=42))
])
text_clf.fit(X_train_text, y_train_labels)
```

(Vectorizes text then trains a logistic regression via SGD) [20]. Evaluate model with `classification_report` or confusion matrix [21] [22]. In 2020, the model's performance on topic prediction was assessed and a discussion of results (e.g. certain classes having lower recall) was expected [23] [24].

- **Hyperparameter Tuning:** Use `GridSearchCV` on the Pipeline to tune parameters like n-gram range or regularization. In 2020, `GridSearchCV(text_clf, parameters, cv=5)` was used to find optimal settings [25]. Time the fitting process if needed (they measured fit time to compare full vs subset of data) [26].

- **NetworkX for Wikipedia Link Graph:** Construct a directed graph of Wikipedia articles and hyperlinks. You can read edges from a TSV and add to a DiGraph. In 2020, edges were added manually for a directed link network [27] [28]. Once built:

- Compute basic stats: number of nodes, edges, and average degree = E/N [29]. The average can be misleading for power-law distributions (better to look at median or log-log plot) [30] [31].
- **Degree Distribution:** Use `G.in_degree()` or `Counter` to get degree frequencies. Plot on log-log scale to see if it follows a power-law (the exam expected recognizing a heavy-tail degree distribution) [31] [32]. Use `plt.loglog(deg, freq)` for visualization.
- **Shortest Paths:** Utilize provided `shortest_path_length` data or compute via `nx.shortest_path_length`. Compare human navigation path lengths vs shortest path lengths

(2020 analysis computed average human path vs optimal) to see if humans take significantly longer routes.

- **Node Centrality:** Compute measures like eigenvector centrality (`nx.eigenvector_centrality`) or in-degree for target nodes, then compare distributions for successful vs failed outcomes. In 2020, they enriched each game record with the target article's in-degree and eigenvector centrality [33], then plotted distributions for games that finished vs not. E.g., higher median in-degree for finished games indicates popular targets are easier to reach.

- **Causal Reasoning:** Frame differences in outcome in a causal diagram. 2020's Task C had students compare success rates for games with high vs low target in-degree: e.g. ~77.8% of high-in-degree target games finished vs ~56.2% for low [34] [35]. A quick bootstrap or significance test (e.g. a two-proportion z-test or computing non-overlapping confidence intervals) can assess significance [35]. They concluded high in-degree targets correlate with higher success. However, a causal diagram was needed to identify confounders (like shortest path length) before attributing causality [36] [37].

2021 Exam (Graph Analytics & Text Bias Analysis)

- **Faculty Hiring Network (Directed MultiGraph):** Build a MultiDiGraph from node and edge lists. Each node is a university with a ranking score attribute, each edge represents a PhD hire (source=PhD university, target=hiring university) with a gender attribute [38] [39]. Use `nx.from_pandas_edgelist(df, 'u', 'v', ['gender'], create_using=nx.MultiDiGraph())` to include multi-edges and attributes [40] [41]. Basic analysis:
- Count nodes and edges: `G.number_of_nodes()`, `G.number_of_edges()` [41].
- **Cumulative Hiring Distribution:** Plot the cumulative fraction of hires by top N universities. In 2021, they calculated what fraction of all hires are made by the top N hiring universities vs produced by top N PhD-source universities [42] [41]. This effectively illustrates *out-degree* vs *in-degree* centrality for nodes. The resulting plot showed the top few universities account for a large fraction of hires (an indicator of hierarchy) [43].

- Identify which centrality metric matches each curve: hiring corresponds to in-degree centrality; producing PhDs corresponds to out-degree centrality [43].

- **Quintile Analysis:** Divide numeric ranks into quintiles with `pd.qcut`. In 2021 Task 2, universities were grouped into 5 quintiles by score, and a **flow matrix** was created showing the fraction of hires from quintile i that go to quintile j [44] [45]. Use a pivot or groupby: count edges where source ∈ Qi and target ∈ Qj, then normalize by total hires from Qi. Visualize this as a heatmap (5x5) to reveal patterns (e.g. a strong diagonal means hires stay within similar rank tiers).

- **Statistical Tests:** Compute differences in means and test significance:

- 2021 Task 3 asked if male vs female hires have different average "score gain" (target_rank – source_rank). Compute mean score gain for men and women separately and perform a two-sample t-test (`stats.ttest_ind`) [46] [47]. For example:

```
diff_m = [score_v - score_u for (u,v,k) in G.edges(data='gender') if
gender=='M']
diff_w = [... same for 'F' ...]
print(np.mean(diff_m), np.mean(diff_w))
stats.ttest_ind(diff_m, diff_w)
```

This yields both means and a p-value [46] [48]. In the solution, men had a slightly higher average score gain and the difference was statistically significant ($p \approx 0.01$) [47] [48].

- **Regression with Controls:** Use `statsmodels` OLS to control for confounders. They ran a linear regression of score gain on gender and source-university score [49] [50]. The statsmodels formula API allows including categorical variables easily. The resulting coefficient for gender (when controlling for source rank) was small and not significant (meaning gender wasn't a significant predictor once rank was accounted for) [51] [52]. Use `model.summary()` to get a full table of coefficients and p-values.

- **Text Data – Gender Bias in Q&A:** The second part of 2021 examined if male vs female tennis players get different questions:

- **Balanced Dataset & Pipeline:** They had a balanced dataset of interview questions (`questions.tsv.gz`) and commentary (`commentary.tsv.gz`) with gender labels. Using scikit-learn, they trained a logistic regression to predict gender from text. They used `TfidfVectorizer` within a Pipeline and `SGDClassifier` for efficiency [53] [20]. They compared model performance on *questions* vs *commentary* texts. The result: questions were more *gender-predictive* than commentary (higher classification accuracy ~71% vs ~60%, indicating possible bias in questions) [54] [55]. Regularization needed tuning: a larger C (weaker regularization) overfit and reduced accuracy on test [56] [57]. Always use an appropriate regularization (smaller C) for sparse high-dimensional data [58] [59].
- **Observational Study:** They computed an aggregate score ("similarity" to tennis-related terms) for each question and ran an OLS regression `similarity ~ gender` [60] [61]. The coefficient was positive, meaning male players' questions were on average more tennis-focused by ~0.25 (and highly significant $p<0.001$) [61]. They then checked if player ranking confounded this (men vs women had slightly different average rankings). They plotted ranking vs question similarity and computed Spearman's ρ to see if higher-ranked players get different questions [62]. It turned out rank and similarity had some correlation, but the gender effect remained strong. Always consider potential confounders (like player rank) when interpreting such differences.

2022 Exam (Signed Networks & Structured Prediction)

- **Signed Graph Triads (Structural Balance):** Model a social graph with positive/negative edges and analyze triads:
- Build an undirected graph from a list of votes (here, Wikipedia RfA votes). Each user is a node; add an edge `u–v` with sign=+ if user *u* supported *v*'s admin request, or sign=– if they opposed [63] [64]. (Multiple votes between the same pair can be ignored or kept as multi-edges if needed.)
- **Enumerating Triangles:** To find all triangles, you can use `nx.enumerate_all_cliques(G)` and filter for size 3 cliques [65]. Count how many triangles fall into each sign configuration: {+,+,+}, {+,+,–},

{+,–,–}, {–,–,–}. In 2022, they iterated through each triangle's edges and sorted the signs into a tuple to categorize [66] [67]. The fractions were then printed:

- ~79.3% all-positive (friends with friends) [68],
- ~0.5% all-negative [68],
- ~13.9% two-plus-one-minus [68],
- ~6.3% one-plus-two-minus [68].

  According to **Structural Balance Theory**, balanced triads are those with an even number of negative edges (so +++, +– –), which in this case were ~85.5% of all triangles [68]. The observation: very few triangles were unbalanced (+ + – or – – –), supporting the theory that social networks avoid unstable configurations.

- **Features for Prediction:** They engineered features to predict an edge's sign. For a given user pair (u voting on v):
  - *PP:* number of common neighbors X where u–X and v–X are both positive edges.
  - *NN:* number of common neighbors where both ties are negative.
  - *PN:* number of common neighbors where one tie is positive and the other negative.
  - *P, N:* also the voter's overall count of positive and negative votes cast (their general disposition). These features were derived from the graph and vote history.
- **Rule-Based Classifiers:** Two heuristic classifiers were defined:
  - **Structural balance rule:** Predict *support* if adding a positive edge yields more balanced triads than a negative edge. Essentially, vote "+" if `PP + NN > PN`, else vote "–". (This comes from: with a + vote, triads with PP or NN neighbors become balanced, with a – vote, PN neighbors become balanced. In 2022, this heuristic had better AUC.)
  - **Weak balance rule:** Under the weaker theory (which counts – – – as balanced too), predict "+" if `PP > PN` else "–" (since NN triads count as balanced either way).
  - Compute AUC for each by assigning a score (e.g. `score = PP+NN – PN` for the structural rule) and using `roc_auc_score(y_true, score)` [69] [70]. In the exam, classifier A (structural rule) outperformed classifier B (weak rule) slightly.
- **Bootstrap Significance:** They performed a permutation test to see if the AUC difference is significant. For example, sample the dataset 200 times, compute AUC difference each time, and check if 0 falls in the middle of the bootstrap distribution [71] [72]. The result confirmed one rule was consistently better.

- **Logistic Regression:** Finally, they trained a logistic model using the features (PP, PN, NN, P, N) to predict the vote sign. After splitting data, they evaluated AUC on a test set. Including graph-derived features substantially improved AUC (from ~0.67 to ~0.80) [73] [74]. This underscores that structural balance features have predictive power in social vote outcomes.

- **NetworkX Tips:** Use `nx.get_edge_attributes(G, 'YEA')` or similar to filter subgraphs by attribute (e.g. only 2004 edges) [75]. Use `G.edge_subgraph(edges_subset)` to create a view of the graph containing only those edges [75]. MultiDiGraphs can have parallel edges; to analyze simple triangles, converting to an undirected simple graph may be necessary (or treat parallel edges as single for triangle counting).

2023 Exam (TV Show Data – Graphs and NLP)

- **Data Wrangling (Friends Transcript):** Load JSON lines into pandas (`pd.read_json('exam1.jsonl', lines=True)`). Clean data by removing non-dialogue

"speakers" like stage directions (e.g. rows where speaker is "TRANSCRIPT_NOTE" or "#ALL#") [76] . Add new columns by parsing strings:

- Extract season and episode from the utterance ID (format `"sAA_eBB_cCC_uDDD"`). For example, `df['season'] = df['conversation_id'].str.split('_').str[0]` to get "s01", etc..
- Compute utterance length (number of characters) with a lambda: `df['length'] = df['text'].apply(len)`.

- **Statistical Analysis of Dialogue:** They asked if dialogue length changes by season. Fit an OLS regression with season as categorical:

```
model = smf.ols('length ~ C(season, Treatment(reference="s01"))',
data=df).fit()
print(model.summary())
```

This provides coefficients for each season relative to season1 [77] [78] . In the output, the intercept = 51.0 (avg length in S1 ~51 chars) [77] . Only a couple of seasons had significant differences: e.g. Season9 coefficient ≈ +6.09 ($p<0.001$, significantly longer lines) [78] , whereas Season10 was not significantly different ($p=0.409$) [79] . Interpret: Season9 episodes have slightly wordier utterances on average, while others are about the same as Season1. The regression summary also answered what the intercept and season coefficients represent (baseline and difference from baseline) [80] [78] and asked whether those differences are significant at 0.05 (look at p-values).

- **Identifying Main Characters:** The exam asked to **visually argue** that there are 6 main characters. A common approach is to plot the cumulative distribution of lines per character (sorted by frequency). In 2023, they computed the ECDF of utterances per speaker: count lines per speaker, sort by frequency, do cumulative sum divided by total [81] [82] . Plotting this with a logarithmic x-axis showed a sharp jump: the top 6 speakers (Rachel, Ross, Monica, Chandler, Joey, Phoebe) account for a large fraction of all lines (the curve rises quickly by speaker rank 6, then flattens) [82] . This "long tail" plot makes it evident there are six protagonists.

- **Conversation Graph:** Create a directed multigraph of who replies to whom:

- Nodes: characters (speaker names). Edges: if character A's line is a reply to character B, add a directed edge A→B. The dataset had `reply_to` fields linking utterances. In pandas, merge the DataFrame with itself to map each utterance's `reply_to` ID to the speaker of the replied-to utterance [83] . Filter to only reply rows (where reply_to is not None) [84] .
- Use `nx.from_pandas_edgelist` on that merged data, with `source='speaker_x'` (replier) and `target='speaker_y'` (original speaker), and include attributes like season/episode [85] . Specify `create_using=nx.MultiDiGraph()` to allow parallel edges [85] .
- The exam then printed the graph's size: e.g. 692 nodes, 54904 directed edges (replies) [86] . They also discussed an alternative to multi-edges: use a weighted edge to count multiple replies between the same two people (so the graph would be a weighted simple DiGraph instead of MultiDiGraph) [87] .

- **Centrality Measures:** Using a provided graph (they distributed a cleaned GraphML), they calculated:

- **Out-degree** for each node (just the count of outgoing edges, not normalized) by `G.out_degree()` [88].
- **PageRank** for each node via `nx.pagerank(G)` with default damping [89].
- Listed these for the six main characters [90]. In the results, Rachel had the highest PageRank (0.1268) and highest out-degree (8470) [91] [92], slightly above Ross [93]. Thus, **Rachel Green** was identified as the most central character by both metrics [94] [95]. Chandler, Monica, Joey, Phoebe followed, confirming all six leads rank top in network importance.

- The exam asked some true/false conceptual questions about PageRank:

    - Inverting all edges *will* change PageRank (True answer: **False** statement, because incoming vs outgoing link structure is reversed, altering prestige) [96] [97].
    - Removing all outgoing edges from Rachel won't leave her PageRank unchanged (that statement is **False** – while PageRank mainly depends on incoming links, dangling nodes can affect overall distribution slightly, and at least the network normalization might change) [96] [97].
    - Adding a new node with 1000 outgoing edges to everyone else (and no one linking back) won't give it the highest PageRank – it will actually get a very low PageRank (it's a dead-end that doesn't receive any links), so the statement "it would have highest PageRank" is **False** [96] [98].

- **Text Similarity & Classification:** Part 2 of 2023 focused on character language uniqueness:

- They prepared a token list for a specific character (Chandler) by aggregating all words he spoke. For instance, gather all distinct tokens Chandler Bing used over all seasons and sort them [99] [100]. This yields vocabulary L.
- Then they built an **episode-level word frequency matrix** for Chandler: each row = an episode, each column = a token from L, and entries = frequency of that token in Chandler's lines that episode. This likely used a sparse matrix approach (scipy or manual loops). The notebook shows a snippet of a matrix row for s01_e01 with many 0.0 entries [101] (possibly a TF-IDF or normalized frequency).

- They extended this idea to compare main characters. Possibly a matrix of characters vs token frequencies or a distance matrix between characters' word usage was computed. The snippet [102] suggests a table where each row is a true character and each column a predicted character, likely from a classifier confusion matrix. Indeed, they likely trained a classifier to predict which main character said a given line based on word features, and printed the confusion matrix (with class labels Chandler, Joey, Monica, Phoebe – perhaps only the four with most distinctive speech). The diagonal values were low (~0.03–0.04), indicating the classifier wasn't very accurate distinguishing them, though Monica/Phoebe might be slightly more distinct (their confusion values differ) [103]. Another approach: compute cosine similarity between characters' TF-IDF vectors; the snippet [104] looks like pairwise cosine similarities between characters (Chandler-Chandler 0.02475 vs Chandler-Phoebe 0.01572, etc.). The values are all close, meaning the characters' overall vocabularies are not drastically different. Any differences might require focusing on specific unique catchphrases or n-grams.

- **Key NLP techniques:** assembling documents per character or episode, vectorizing text (word counts or TF-IDF), and optionally training a classifier or computing similarity. They likely used scikit-learn's `TfidfVectorizer` for character classification as well.

2024 Exam (Negotiation Chats & Outcome Prediction)

- **Document Aggregation:** They combined turn-by-turn negotiation data into per-negotiator documents. Each negotiation had two agents (mt_agent_1 and mt_agent_2); for each agent in each negotiation, all their message texts were concatenated into one document (joined with newline) [105] [106]. In pandas this was done by grouping:

```
df_document = df_negotiations.groupby(['negotiation_id','agent'])
['message'] \
                 .apply(lambda msgs:
'\n'.join(msgs)).reset_index(name='document')
```

(Thus each "document" is everything one person said in one negotiation) [107]. They then merged this with a metadata table (`df_meta`) containing each negotiation's outcome and participant info (age, gender, etc.) [108]. The merged DataFrame had one row per **(negotiation, agent)** with their concatenated dialog and outcome score [109] [110]. (They printed the merged size 12,336 and a preview to verify merge success.)

- **TF-IDF Feature Extraction:** Using the documents above, they computed a TF-IDF matrix to analyze language. They used `sklearn.feature_extraction.text.TfidfVectorizer` with `max_features=100` (limit to top 100 terms) and `stop_words='english'` to drop common words [111] [112]. The result `tfidf_matrix` has shape (N_documents, 100) [111] [112]. Each row is a negotiator's combined dialogue represented in a 100-dimensional TF-IDF vector.

- **Success vs Failure Label:** They created a binary label "success" for each document, defined as 1 if the negotiation outcome > median, else 0 [113]. For example:

```
median_outcome = df_document['outcome'].median()
df_document['success'] = (df_document['outcome'] >
median_outcome).astype(int)
print("median outcome:", median_outcome)
```

This splits negotiations into roughly half successful, half not [113] [114] (the median outcome was printed for reference).

- **Train/Test Split and Logistic Regression:** They built a model to predict success from the TF-IDF features:

- Defined `X = tfidf_matrix` and `y = success` labels [115].
- Split into training and testing sets (20% test, random_state=99 for reproducibility) using `train_test_split(X, y, test_size=0.2, random_state=99)` [116].
- Trained a logistic regression classifier (likely with default parameters) on the training set and made predictions on the test set. They then printed a classification report (`precision`, `recall`, etc.) [117] [118]. The report in the solution shows an **imbalance**: the model achieved high recall (0.94) for

class 0 (unsuccessful negotiations) but very low recall (0.04) for class 1 (successful) [119]. Overall accuracy was ~62% [117], but the model mostly predicts "unsuccessful" for most cases (class imbalance issue).

- This highlights the need for techniques like class weighting or more features to improve predicting the relatively rarer class (success).

- **Chi-Square and Feature Insights:** The solution suggests they performed a chi-square test on a contingency table, likely comparing some categorical feature usage between success vs failure [120]. For example, they might have picked a specific word from the TF-IDF features (e.g. usage of "please" or "thank") and built a table of how often that word appears in successful vs unsuccessful docs, then used `scipy.stats.chi2_contingency` to see if the difference is significant [120]. No specific outcome was given in the snippet, but chi-square is a common test for independence in such categorical data.

- **IDF Interpretation:** A discussion point was that using TF-IDF (which down-weights common terms) helps highlight distinctive words used by successful vs unsuccessful negotiators [121]. High TF-IDF terms in successful negotiations might point to strategies or tone (e.g. successful negotiators might use more cooperative language). The **IDF component** ensures that very common words (even if frequent in successful chats) don't overshadow truly distinctive terms.

- **Potential Bias – Message Length:** They cautioned that successful negotiators might simply talk more (longer messages), which could inflate their TF-IDF scores across the board. If word count is higher for successes, many words (even unimportant ones) get higher TF counts, which IDF might not fully offset. The exam asked how to test for this bias [122] [123]. A simple check: compute the correlation between total message length and outcome, or compare average lengths for success vs failure. The solution suggests normalizing or at least examining this relationship [124]. For instance, one could include a control variable for message length or use length as a feature to see if it alone predicts success. If successful outcomes correlate strongly with length, the NLP model might be picking up length rather than specific word choices. A remedy could be to normalize TF-IDF by document length or use length as a covariate to isolate vocabulary effects.

---

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 exam_2019.ipynb
file://file-CrGGwhPHP8LNhammXYvTFc

20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 53 exam_2020.ipynb
file://file-CQV9HkzHCwVJdCiGv9BXtz

38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 54 55 56 57 58 59 60 61 62 exam_2021.ipynb
file://file-9bmUGxamWa697WSUyv6G4V

63 64 65 66 67 68 69 70 71 72 73 74 75 exam_2022.ipynb
file://file-RjFP8m6EEELZiAcw5hHMXK

76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104
exam_2023.ipynb
file://file-Qrs8z9j7GpuELUSpgmi3Jm

105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 exam_2024.ipynb
file://file-MFEWBHtfSY5qMS4aBNuSVW